

AI Revenue Recovery Agent — Flowchart & System Map

Autonomous Omni-Channel Financial Pipeline · Razorpay Gateway · Groq Llama-3.3-70b · Express & MongoDB

MASTER
BLUEPRINT

PHASE 1: STOREFRONT SELECTION & CHECKOUT INITIATION

1. Course Browsing

`Courses.js / Batches.js`

Student selects batch (e.g. JEE Mains 2026).

2A. Free Batch (Price = 0)

`POST /api/save-purchase`

Direct enrollment & instant redirect to `/class/:id`.

2B. Paid Batch (e.g. ₹4,999)

`POST /api/create-order`

Razorpay Order created & PaymentAttempt logged.

3. Razorpay Popup Modal

`new window.Razorpay(opts)`

Opens secure payment gateway modal.

↓ (If Payment Fails / OTP Timeout / Modal Dismissed)

PHASE 2: DUAL INGESTION & REAL-TIME FAILURE CAPTURE

Browser Fast-Path Handler

`recoveryClient.js → POST /api/recovery/payment-failed`

Client reports failure instantaneously.

Official Razorpay Webhook

`webhook.js → POST /webhook/razorpay`

HMAC-SHA256 verified payment.failed event.

Signal Snapshot Created

`FailedPayment.model.js`

Stored with status: "open", attempts: 0.

↓ (Signal sent to Cognitive Layer)

PHASE 3: AI REASONING & POLICY GATE GOVERNANCE

1. Orchestrator Controller

`orchestrator.js → processSignal()`

Coordinates reasoning & gate evaluation.

2. AI Reasoning Engine

`reasoningService.js` (Groq Llama-3.3-70b)

Diagnoses root cause & proposes 10% discount.

3A. Auto-Approved (≤ ₹5,000)

`gateService.js → decision: "approved"`

Autonomous discount voucher creation.

3B. Escalated (> ₹5,000)

`gateService.js → decision: "pending"`

Routed to Human-in-the-Loop Admin Queue.

↓ (When Action Approved)

PHASE 4: VOUCHER MINTING & MULTI-CHANNEL TOUCHPOINT LINKS

1. 24h Expiring Voucher

`RecoveryOffer.model.js`

Locks ₹4,499 price with 24h TTL.

2. In-App Notification

`Notification.model.js`

Popup: "10% DISCOUNT approved!"

3. WhatsApp / SMS Nudge

`whatsappRecovery.js`

Link: `/pay-discount/:offerId`.

4. Instant Screen Modal

`DiscountRecoveryModal.js`

Pops up immediately without refresh.

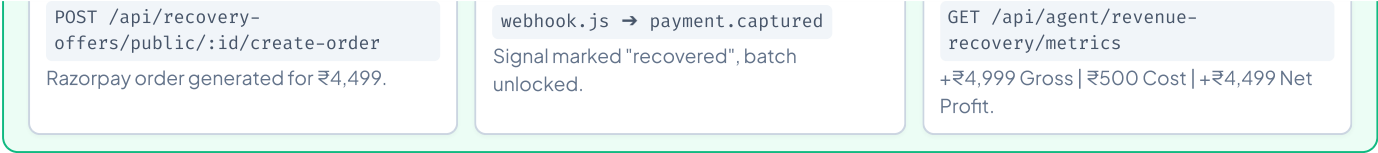
↓ (Student Pays Discounted Amount)

PHASE 5: DISCOUNT CHECKOUT, COURSE UNLOCK & FINANCIAL ATTRIBUTION

1. Discount Order Created

2. Verified Webhook

3. Financial ROI Ledger



Technical Specifications & Route Mapping

Every File, Route, Schema, and External API Call Explained in Execution Order

TECHNICAL SPECS

#	Action & Purpose	Frontend File & Trigger	API Request & Route	Backend File, Schema & API Used
1	Browse & Select Batch	<code>Courses.js</code> Click "Enroll Now"	POST <code>/api/create-order</code>	<code>back/index.js</code> (Line 238) Schema: <code>PaymentAttempt.model.js</code> API: Razorpay Orders API
2	Launch Payment Modal	<code>Courses.js</code> <code>new</code> <code>window.Razorpay(opts)</code>	Internal SDK initialization	Client-side Razorpay Gateway Popup iframe rendering.
3	Report Payment Failure	<code>recoveryClient.js</code> <code>rzp.on('payment.failed')</code>	POST <code>/api/recovery/payment-failed</code>	<code>recoveryClient.routes.js</code> (Line 17) Schema: <code>FailedPayment.model.js</code>
4	HMAC Webhook Ingestion	Razorpay Cloud Server	POST <code>/api/agent/revenue-recovery/webhook/razorpay</code>	<code>webhook.js</code> (Line 18) Schema: <code>FailedPayment.model.js</code> Crypto: <code>crypto.createHmac('sha256')</code>
5	AI Root Cause Reasoning	Background Orchestrator	Internal <code>processSignal(signalId)</code>	<code>reasoningService.js</code> (Line 139) Schema: <code>AgentAction.model.js</code> API: Groq Llama-3.3-70b / OpenRouter
6	Policy Gate Governance	Background Orchestrator	Internal <code>evaluateGate()</code>	<code>gateService.js</code> (Line 5) Schema: <code>AgentPolicy.model.js</code> (Cap: ₹5,000)
7	Voucher & Links Minting	Background Dispatcher	Internal <code>issueRecoveryOffer()</code>	<code>recoveryOfferService.js</code> Schemas: <code>RecoveryOffer</code> , <code>Notification</code>
8	Instant Popup on Screen	<code>DiscountRecoveryModal.js</code> CustomEvent listener	Client-side DOM event: <code>recovery-discount-offer</code>	Displays 10% discount popup (~~₹4,999~~ → ₹4,499) with 24-hour countdown timer.
9	Create Discount Order	<code>DiscountRecoveryModal.js</code> Click "Pay ₹4,499"	POST <code>/api/recovery-offers/public/:id/create-order</code>	<code>recoveryOffer.routes.js</code> (Line 33) API: Razorpay Orders API
10	Verified Attribution & Unlock	Razorpay Cloud Server	POST <code>/api/agent/revenue-recovery/webhook/razorpay</code>	<code>webhook.js</code> (Line 103) Marks <code>status = "recovered"</code> & updates <code>/metrics</code> .

- ### 1. Ingestion & Dual Fast-Path Reporting

When an OTP fails or the student dismisses checkout, `recoveryClient.js` posts to `/api/recovery/payment-failed`, while Razorpay sends an HMAC-signed webhook. This dual capture ensures 0% signal loss.

Files: `recoveryClient.js`, `webhook.js` **Security:** HMAC-SHA256 **Status:** open
- ### 2. AI Diagnostic Cognition (Groq Llama-3.3-70b)

Groq's Llama-3.3-70b processes the failure in ~250ms, diagnoses the cause (`bank_otp_timeout`), calculates confidence (0.95), and writes a personalized Hinglish voice script offering a 10% discount.

File: `reasoningService.js` **Fallback:** OpenRouter Nemotron / Deterministic Matrix
- ### 3. Policy Gate (Auto-Approve vs HITL Queue)

Signals \$≤ ₹5,000\$ are auto-approved. High-value transactions (\$> ₹5,000\$) are routed to the Human-in-the-Loop Admin Queue (`/approval-queue`) for mandatory 1-click admin sign-off.

File: `gateService.js` **Rules:** 10% max discount, 3 retries limit, 15m cooldown

4. Verified Recovery & Real-Time Financial ROI

When the student pays via the 24-hour voucher, Razorpay confirms the transaction. The agent attributes the revenue, updates the Net Profit margin (`Gross - Discount`), and unlocks course access instantly.

File: `webhook.js, routes.js (/metrics)` **Result:** +₹4,999 Gross | ₹500 Discount | +₹4,499 Net Profit